Task 3 - Secure Coding Review

NIVIA SHERIN. V

II M.SC CYBERSECURITY

CODE ALPHA INTERN

INDEX

S.No	Section Title	Page No
1	Introduction	3
2	Application Reviewed	3
3	Identified Vulnerability	4
4	Security Risk Analysis	5
5	Secure Code Implementation	5
6	Program Execution and Testing	6
6.1	Execution of Vulnerable Program	7
6.2	Command Injection Demonstration	8
6.3	Secure Program Execution	8
7	Secure Coding Best Practices	9
8	Conclusion	10
9	References	10

Task 3 - Secre Coding Review

Secure Coding Review: Command Injection Vulnerability Analysis

1. Introduction

Secure coding practices are essential for developing software applications that are resilient against cyber threats and security vulnerabilities. Many security incidents occur because developers unintentionally introduce weaknesses in the code, such as improper input validation or unsafe system command execution. These vulnerabilities can be exploited by attackers to gain unauthorized access, execute malicious commands, or compromise sensitive data.

A secure coding review is a systematic process used to analyze application source code in order to identify potential security flaws and implement corrective measures. By reviewing the code and applying secure programming principles, developers can significantly reduce the risk of cyber-attacks.

This project demonstrates a secure coding review of a simple Python application. The objective of the review is to identify security vulnerabilities in the code, analyze the associated risks, and implement a secure solution to mitigate the identified issue.

2. Application Reviewed

The application reviewed in this project is a simple Python-based command-line program designed to accept input from a user and display a greeting message. The program uses the os.system() function to execute an operating system command that prints the user's name.

The purpose of this program is to demonstrate how improper handling of user input can introduce security vulnerabilities into an application. Because the program directly incorporates user input into a system command without any validation or sanitization, it creates an opportunity for attackers to manipulate the command execution process. This implementation highlights a common programming mistake where unvalidated user input is directly executed by the system. Such practices can introduce serious security vulnerabilities and expose the application to potential command injection attacks.

The **vulnerable code implementation** is shown below:

```
import os

print("Secure Coding Review Demo")

user_input = input("Enter your name: ")

os.system("echo Hello " + user_input)
```

In this implementation, the user input is directly appended to a system command. Since the input is not validated, malicious users can inject additional commands that may be executed by the operating system.

3. Identified Vulnerability

The primary vulnerability identified in this application is Command Injection. Command injection occurs when an application allows user-controlled input to be executed as part of a system command. This type of vulnerability arises when user input is not properly validated or sanitized before being passed to system-level functions.

In the vulnerable program, the os.system() function is used to execute a shell command that includes user input. Because the application does not verify or restrict the input provided by the user, an attacker can inject additional commands into the input field.

For example, the attacker may enter the following input:

```
Sherin & dir
```

Instead of executing only the intended command, the operating system interprets this input as multiple commands. As a result, the program executes the dir command and displays the contents of the directory.

This behavior demonstrates how attackers can exploit command injection vulnerabilities to execute unauthorized commands on the system. In real-world scenarios, such vulnerabilities can be used to access sensitive data, modify files, or gain control over the affected system.

4. Security Risk Analysis

Command injection vulnerabilities pose significant security risks to software systems. When attackers are able to inject and execute unauthorized commands, they may gain control over the operating environment of the application.

In the reviewed program, the use of the os.system() function allows user input to be directly interpreted by the operating system. This means that any additional command included in the input may be executed by the system.

If exploited in a real-world application, command injection attacks could allow attackers to perform actions such as:

- Executing unauthorized system commands
- Accessing or modifying sensitive system files
- Installing malicious software or backdoors
- Disrupting normal system operations
- Compromising the confidentiality and integrity of data

Because of these potential consequences, command injection vulnerabilities are considered high-severity security flaws and must be mitigated through secure coding practices.

5. Secure Code Implementation

To address the identified vulnerability, the insecure os.system() function was replaced with a safer implementation using the subprocess.run() function.

The secure implementation ensures that user input is treated strictly as data rather than as part of a system command. This approach prevents attackers from injecting additional commands into the execution process. These changes improve the overall security of the application by eliminating the unsafe execution of user-controlled input. Implementing secure coding techniques helps ensure that the program processes user input safely without allowing unintended command execution.

The improved **code implementation** is shown below:

```
import subprocess

 print("Secure Coding Review Demo")

user_input = input("Enter your name: ")

subprocess.run(["cmd", "/c", "echo", "Hello", user_input])
```

In this version of the program, the command and its arguments are passed as a list to the subprocess.run() function. This prevents the shell from interpreting special characters in user input as separate commands.

By separating command arguments and avoiding direct shell execution, the risk of command injection is significantly reduced.

6. Program Execution and Testing

To evaluate the vulnerability and verify the effectiveness of the secure implementation, the application was tested using both normal input and malicious input.

Test Case 1 – Normal Input

When a normal input such as “Sherin” is entered, both the vulnerable and secure programs produce the expected output:

```
Hello Sherin
```

This confirms that the program performs its intended function under normal conditions.

Test Case 2 – Command Injection Attempt

A malicious input was provided to simulate a command injection attack:

```
Sherin & dir
```

In the vulnerable version of the program, the operating system executed the additional dir command and displayed the contents of the directory. This confirms that the application is vulnerable to command injection.

However, when the same input was tested in the secure version of the program, the input was treated purely as text and no additional command was executed.This demonstrates that the secure implementation successfully prevents command injection attacks.

6.1 Execution of Vulnerable Program

To demonstrate the behavior of the vulnerable application, the program was executed with normal user input. The following screenshot shows the successful execution of the vulnerable program when a standard input value is provided.

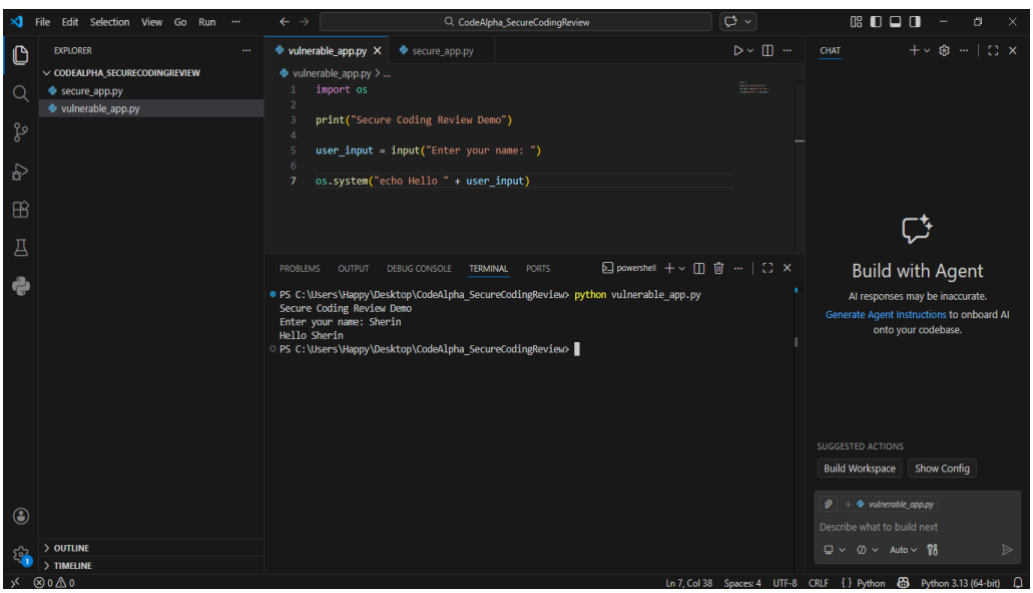

Figure 1: Execution of the vulnerable application with normal input.

The vulnerable Python program is executed using the command python **vulnerable_app.py.** When the user enters a normal input such as **"Sherin"**, the program displays the expected output message. In this scenario, the application appears to function correctly because the input does not contain any malicious commands.

However, the program directly executes user input using the os.system() function, which creates a potential security risk.

6.2 Demonstration of Command Injection Attack

To analyze the security weakness in the application, a command injection attempt was performed by providing malicious input. The following screenshot illustrates how the vulnerable program executes unintended commands due to improper input handling.

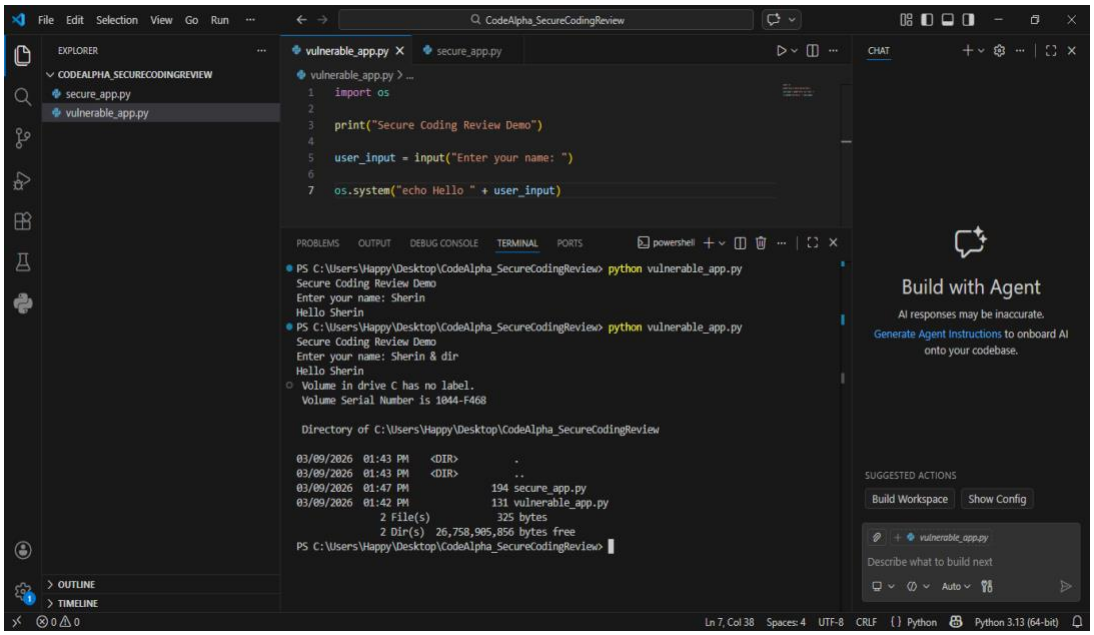

Figure 2: Demonstration of command injection vulnerability.

In this test, a malicious input **"Sherin & dir"** is provided to the application. Because the program directly passes user input into the system command, the operating system interprets the additional command dir and executes it.

As a result, the directory contents of the project folder are displayed. This demonstrates how an attacker can inject and execute unintended commands through the application, confirming the presence of a command injection vulnerability.

6.3 Execution of Secure Application

After implementing the secure coding solution, the program was tested again using the same malicious input. The following screenshot demonstrates that the secure implementation prevents command injection and safely processes user input.

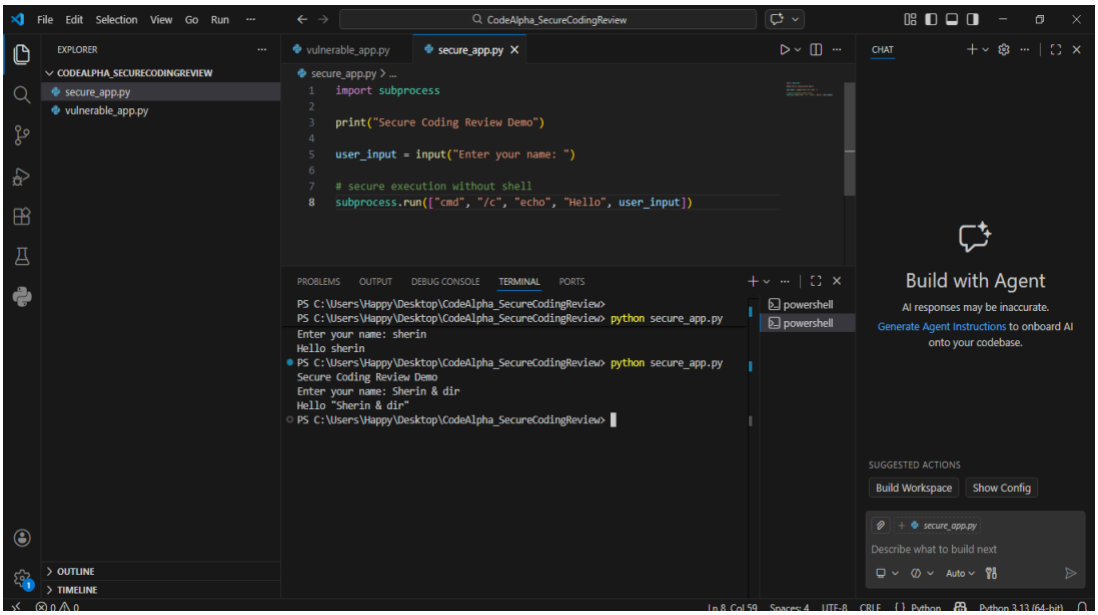

Figure 3: Execution of the secure version of the application.

The secure implementation replaces the insecure os.system() function with the safer subprocess.run() method. In this version, user input is passed as an argument to the command rather than being directly interpreted by the system shell.

When the same malicious input **"Sherin & dir"** is provided, the application treats it as plain text and simply displays it as part of the output. The additional command is not executed, demonstrating that the secure implementation effectively prevents command injection.

7. Secure Coding Best Practices

To prevent similar vulnerabilities in software applications, developers should follow secure coding practices throughout the development lifecycle. Some important best practices include:

- Validate and sanitize all user inputs before processing them
- Avoid executing raw system commands directly from application code
- Use secure libraries and built-in functions that prevent command injection

- Implement proper error handling and input restrictions
- Conduct regular security reviews and vulnerability assessments

Following these practices helps developers build applications that are more resilient to cyber attacks.

8. Conclusion

This project demonstrated the process of conducting a secure coding review on a Python application containing a command injection vulnerability. The analysis identified how the improper use of system command functions can expose an application to serious security risks.By replacing the insecure implementation with a safer method and applying secure coding principles, the vulnerability was successfully mitigated. The results highlight the importance of secure programming techniques and regular code reviews in preventing potential cyber threats.Secure coding practices play a critical role in ensuring the reliability, security, and integrity of modern software systems.

9. References

(I) OWASP Foundation – Command Injection Vulnerability https://owasp.org/www-community/attacks/Command_Injection

(II) Python Documentation – subprocess module https://docs.python.org/3/library/subprocess.html

(III) OWASP Secure Coding Practices Guide https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/